

Report Structures

Three reports, one per run (or two, when there is no legacy codebase). Each is a **single self-contained HTML file**: no CDN, no external fonts, no network, no server. It opens from disk in any browser, prints to PDF cleanly, attaches to an email, and can be committed to a repository.

Report	Produced when	Remediation	Answers
Legacy	a legacy path was given	no	What was wrong before the migration?
Modernized	always	yes , full	What is wrong now, and how do we fix it?
Comparison	both audits succeeded	yes , carried over	What did the migration <i>do</i> to us?

```
<out>/<Project name>/
├── <Project> - Legacy - Security analysis report. - YYYY-MM-DD.html
├── <Project> - Modernized - Security analysis report. - YYYY-MM-DD.html
└── <Project> - Comparison - Security analysis report. - YYYY-MM-DD.html
```

The period after "report" is part of the required filename format. Only characters illegal in

filenames (`\`/: * ? " < >	) are stripped from the project name.
----------------------------	---------------------------------------

Common to every report

How it is built

```
assets/*.part1.html  +  <data>.json  +  assets/*.part2.html  =  the report
  stylesheet          the analysis    the renderer
```

The model authors **only the JSON**. The template renders it. Consequences worth knowing:

- Every report in the folder is visually identical in structure — they are one document set.
- Escaping is handled in one place, so a code excerpt containing `<script>` cannot break a report.
- If the JSON is malformed, the page shows an explicit error card naming the parse error, rather than a silently wrong or blank report.
- The JSON stays on disk in `.security-audit/`, so the analysis is machine-readable as well as human-readable.

Interactive behaviour

Control	Effect
Theme	Light / dark toggle, remembered per report via <code>localStorage</code>
Severity chips	Filter findings by severity; multiple chips combine

Control	Effect
Expand / collapse all	Opens every visible finding at once
Deep links	Every finding has a stable <code>#id</code> ; linking to one opens and scrolls to it

The audit reports add a **search box** over finding text and a **Discovered / Remediated checklist** whose ticks persist in `localStorage` , so the report works as a live worklist. The comparison report adds **bucket chips** alongside the severity chips.

Colour never carries meaning alone. Every severity and every bucket ships an icon or a text label next to its colour, and the category charts are mirrored by a data table — which is also the form that prints.

Print

The stylesheet has a print block: the toolbar and theme toggle are hidden, all findings are forced open, and cards avoid breaking across pages. Browser "Print to PDF" is the supported PDF path — there is no separate PDF generator to install.

Accessibility and portability constraints

- Responsive to roughly 400px; tables scroll horizontally inside their own containers rather than forcing the page to.
- Works from `file://` . `localStorage` failures (private windows, blocked site data) are caught, and the report renders correctly without persistence.

1. Legacy report

```
<Project> - Legacy - Security analysis report. - YYYY-MM-DD.html
```

A **diagnostic baseline**. It establishes what was wrong in the system being replaced, so the comparison can say what the migration did about it. Produced with `--mode analyze --variant legacy` .

Sections

#	Section	Contents
—	Masthead	Project name with a <code>Legacy</code> badge, target path, date, depth, stack, scope counts
1	Score dashboard	Current score /100, grade, posture band, and counts by severity
2	Category breakdown	Per-category score, weight, OWASP mapping; bar chart plus data table
3	Controls verified present	Strengths — controls found working. Keeps the score credible rather than alarmist
4	Findings summary	Sortable/filterable index of all findings
5	Detailed findings	One expandable card per finding — see below
6	Analysis cost	Tokens and dollars for this phase, by model
7	Scan evidence	What was checked, with query and result. Distinguishes <i>assessed and clean</i> from <i>not assessed</i>
8	Limitations & scope	What was not done — including the deliberate absence of remediation

What is deliberately absent

Absent	Why
Per-finding remediation	The system is being replaced. A fix plan invites effort on the wrong codebase
Verification steps	Same
Remediation roadmap	Same
Projected score and uplift	There is no plan to project against

The Limitations section states this explicitly:

"Remediation guidance was intentionally not produced for this codebase; this report is a diagnostic baseline only."

A reader must never have to wonder whether the fixes were omitted on purpose or forgotten.

A finding card

► S-3	[High]	Customer search builds SQL by concatenation
Standards	A03:2021 Injection · CWE-89 · ATT&CK T1190 · SSDF PW.5	
Risk	What an attacker gets, in plain terms	
Evidence	What was actually observed, and where	
Locations	custlook.4gl:812 + real code excerpt	
Impact	Confidentiality High · Integrity High · Availability Low	

Every finding carries a **confidence** — `Confirmed`, `Likely` or `Possible` — set by reading the code, not by the pattern that flagged it. Anything that cannot reach `Confirmed` or `Likely` is dropped or filed as `Info` with the open question stated.

2. Modernized report

<Project> - Modernized - Security analysis report. - YYYY-MM-DD.html

The **actionable** report: everything in the Legacy report, plus the whole remediation apparatus. Produced with `--mode plan --variant modernized`. This is also exactly what a standalone `/security-audit` run produces.

Sections

Same as Legacy, with three additions:

#	Section	Contents
1	Score dashboard	Current and projected score, plus the uplift if the plan is applied
5	Detailed findings	Each card adds Remediation, Verification and a Discovered/Remediated tick

#	Section	Contents
6	Remediation roadmap	Phases ordered by risk reduction per unit of effort, with the projected score after each

A finding card, with remediation

► S-1 [Critical] No authentication anywhere in the application

Standards	A01:2021 · CWE-862 · CWE-306 · T1190 · AC-3 · IA-2
Risk	...
Evidence	0 [Authorize] across 148 files; no UseAuthorization
Locations	Program.cs:22 + excerpt
Impact	C High · I High · A Low
Remediation	[Non-breaking] [Effort M] 1. Register the identity provider ... 2. Set a global fallback policy – no per-page edits

Before

app.MapBlazorHub();

After

app.UseAuthentication();

Verification Unauthenticated GET /customers returns 302. Build: 0

☐ Discovered ☐ Remediated

The non-breaking principle

Every remediation is written to **preserve behaviour for legitimate users**: additive middleware, attributes, configuration, opt-in validation. Where a fix genuinely cannot be non-breaking it is marked **Breaking change** with a visible warning and a migration note — never presented as a drop-in.

Every remediation carries a **concrete verification step** the engineer can actually run. A fix without a way to confirm it is a suggestion, not a remediation.

Score dashboard

Current
53 / 100
D- At Risk

Projected after remediation
90 / 100
A- Hardened

Uplift
+37
points

The projected score is labelled *projected*, never *achieved*. It assumes every planned fix is applied **and verified**; findings with no concrete fix, and unapproved breaking changes, stay at full penalty in the projection.

The score is auditable

Weights are fixed and published in `references/scoring.md`, not tuned per project. Categories start at 100 and lose points per finding, scaled by confidence:

```
categoryScore = max(0, 100 - Σ (basePenalty × confidenceFactor))
overall       = Σ (categoryScore × weight) / Σ (weight)      -- assessed categories only
```

A category that was **not assessed** scores `null`, is excluded from the denominator, and appears as "Not assessed" — never scored as 0 or 100. Posture bands have overrides: one Critical finding forces **At Risk** regardless of the number, because an application with an unauthenticated administrative endpoint is not "Hardened" at 91.

3. Comparison report

`<Project> - Comparison - Security analysis report. - YYYY-MM-DD.html`

The one that answers the question the delivery team actually has. Produced only when both audits succeeded.

Sections

#	Section	Contents
—	Masthead	Both stacks, both paths, date, depth, findings compared
0	Verdict	One paragraph: what the migration actually did. Written last
1	Security posture, before and after	Legacy score, Modernized score, migration delta, projected
2	What the migration did	Five bucket tiles + severity movement table
3	Category scores, legacy vs modernized	Two-tone bars per category, plus the delta table
4	Resolved, or simply not migrated?	Coverage risks — the section most likely to be news
5	Finding-by-finding comparison	Side-by-side cards, filterable by bucket and severity
6	Remediation roadmap	Phased plan for the modernized codebase
7	Cost of this analysis	All three phases and the run total
8	How the comparison was made	The pairing method, so a reader can judge it
9	Limitations	Both audits' limitations plus the comparison's own

Posture dashboard



Neither score is recomputed — each audit owns its number, and the comparison joins them. A third figure computed here would agree with neither report in the same folder.

When exposure differs between the two systems, a caveat renders directly under the tiles. A private-network terminal application and a public web application are not judged against the same threat model even when the arithmetic is identical, and equal severities across the two columns do not imply equal risk.

The five buckets

Introduced 1 Created by the modernization	Inherited 1 Present before and after	New surface 1 New architecture, new exposure	Resolved 2 Present in legacy, gone in modernized	Not comparable 1 No counterpart existed to compare
---	--	--	--	--

Items are ordered **Introduced** → **Inherited** → **New surface** → **Resolved** → **Not comparable**, then by severity. The order is the order a team should act in, not alphabetical.

`introduced` and `new-surface` are kept separate deliberately. A 4GL terminal application has no CSRF risk, no CORS policy and no cookie flags; finding those gaps in the web rewrite is the cost of the new architecture, not a regression. Filing every modernized-only finding as "introduced by the migration" produces a document that reads as an indictment and gets dismissed for that reason.

A comparison card

► [Critical] ●Inherited C-2. Customer search builds SQL by concatenation

StandardsA03:2021 · CWE-89 · T1190 · Customer record CRUD

What the migration did
Parameterized 11 of 12 legacy queries via EF Core – a real improvement.
This one was ported as raw SQL for the LIKE wildcard and the concatenation came with it. High → Critical because the endpoint is now anonymous and internet-facing rather than behind the terminal sign-on.

Evidence on each side

LEGACY

L-2 ▲High Confirmed
custlook.4gl:812
LET stmt = "SELECT * FROM
customer WHERE name LIKE...

MODERNIZED

M-2 ●Critical Confirmed
Services/CustomerService.cs:47
FromSqlRaw(\$"... LIKE '%{term}%')"

Match: Certain – same CWE-89 in the same search routine; the 4GL concatenation was translated construct-for-construct.

Risk · Remediation [Non-breaking][Effort S] · Before/After · Verification

The side-by-side evidence is the point of the report — both columns always render, and the column with no counterpart says so explicitly ("Not present in the legacy codebase") rather than being omitted.

Every pairing publishes its match confidence. `Certain` / `Probable` / `Tentative`, with the reasoning beside it. A `Tentative` badge is a feature: it tells a reviewer exactly where to check the work. Findings that could not be paired above `Tentative` are left unpaired rather than forced together.

Coverage risks — the section that earns the report

Every `resolved` item is interrogated: **fixed, or simply not migrated?**

Reason	Treatment
<code>fixed-by-construction</code>	A real win. The framework makes the flaw impossible
<code>explicitly-remediated</code>	A real win. Someone deliberately addressed it
<code>feature-not-migrated</code>	Not a win — raised here as an open migration item
<code>not-applicable</code>	Neutral

Operator sign-on —
signon.4gl authenticated every operator before any screen opened.
Nothing equivalent exists in the modernized tree: no login page, no identity provider, no session concept. Two legacy findings disappear with this code — but so does the control itself.

Legacy findings that vanish with it: C-4
Carry forward: treat sign-on as an open migration item, not a resolved risk. Design its replacement before exposure, and do not reproduce the legacy MD5 in the new credential store.

Without this section, a reader skimming "2 resolved" sees two fixes where one is an unbuilt feature whose vulnerability returns the moment it ships.

Which items carry remediation

Bucket	Remediation
<code>inherited</code> , <code>introduced</code> , <code>new-surface</code>	Yes — carried verbatim from the Modernized audit
<code>resolved</code>	No. There is nothing to fix
<code>not-comparable</code>	Only if the finding exists in the modernized tree

Remediation is copied from the Modernized report rather than rewritten. Two differently-worded fixes for one defect, in two reports in the same folder, force the reader to work out whether they differ on purpose.

Cost reporting

Every report carries tokens **and** dollars. This was a hard requirement, and it is treated as a first-class output rather than a footnote.

Audit reports show their own phase, broken down by model:

Model	Input	Cache write	Cache read	Output	Total tokens	Cost
claude-opus-5	1,980	96,400	1,840,220	58,310	1,996,910	\$2.99

The comparison report shows all three phases and the run total, so a reader who opens only that file still sees what the whole review cost — the figure anyone approving the next one needs:

Phase	Total tokens	Cost
Legacy audit	1,357,000	\$2.12
Modernized audit	1,996,910	\$2.99
Comparison	668,660	\$1.12
Total	4,022,570	\$6.24

Honesty rules

`cost.source` is one of three values, and the template renders each differently:

Value	Meaning	Rendering
<code>transcript</code>	Measured from real usage records	The figures, plainly
<code>estimate</code>	Measurement failed; this is an estimate	A visible "estimated" caveat
<code>unavailable</code>	Not measured	"Not collected" — no numbers at all

An estimate is never labelled as measured. If **any** phase was estimated, the whole comparison block is marked `estimate`. A phase that could not be measured keeps its row, greyed and marked not collected, so the gap is visible rather than silently absent.

Dollar figures use published list rates and exclude enterprise, Batch API and partner-platform discounts — stated in the report, because an unqualified dollar figure invites a comparison against an invoice that will not match.

Limitations, in every report

The section an experienced reader checks first, and the one that separates a security report from a marketing document. It states what was **not** done:

- Static analysis only — nothing was built, deployed or exercised at runtime.
- Whether dependency advisories were verified against a live database, or only inventoried.
- Which categories were **not assessed** — these are excluded from the score rather than scored as clean.
- Files and directories skipped, with counts and reasons.
- For the Legacy report: that remediation was intentionally omitted.
- For the Comparison: how many pairings are `Tentative`, which functional areas could not be mapped, and that exposure differs between the two systems.

The governing rule across all three reports: **"no findings in category X" is reportable only if X was actually checked.** Otherwise it is "not assessed".